CÉGEP De Chicoutimi

LIVRABLE 1 : CAHIER DES CHARGES

JEU GO - Dév. App. Avancé | 420-D60-CH | Gr. 01

Henri Pitre

2026-04-10

TABLE DES MATIÈRES

TABLE DES MATIÈRES 2
1. CARACTÉRISATION DU CLIENT 5
1.1 Identification du client 5
1.2 Profil des utilisateurs 5
1.3 Contexte organisationnel 6
2. BESOIN ET CONTEXTE 7
2.1 Origine de la demande 7
2.2 Problématique entraînant le besoin 7
2.3 Cas d'utilisation type 7
2.4 Liste de besoin explicites 8
2.5 Bénéfices attendus 8
3. ENVIRONNEMENT INFORMATIQUE 9
3.1 Plateforme cible 9
3.2 Spécifications minimales du système 9
3.3 Environnement de développement 9
4. ACTEURS DU SYSTÈME 10
4.1 Utilisateur 10
5. FONCTIONNALITÉS DU SYSTÈME 11
5.1 Gestion des participants 11
5.2 Gestion des parties 11
5.2.1 Créer une partie 11
5.3 Gestion du plateau 12
5.4 Gestion du score et des règles 12
5.4.2 Gestion des captures 13
5.4.2 Gestion des captures 13
5.4.3 Gestion des règles de jeu 14
5.4.4 Gestion du suicide 14
5.4.5 Fin de partie 15
5.4.6 Affichage du résultat 15
5.5 Gestion du temps 16

5.5.1 Chronomètre des joueurs 16
5.5.2 Temps total de la partie 16
5.5.3 Sauvegarde des temps (en même temps que d'autres données) 16
5.6 Gestion des données 17
5.6.1 Sauvegarder les données 17
5.6.2 Récupération des données 17
6. FAISABILITÉ DES SOLUTIONS PROPOSÉES 18
6.1 Gestion de la règle du Ko 18
6.1.1 Solution A – Ko simple 18
6.1.2 Solution B – Superko 19
6.2 Gestion des règles de score 19
6.2.1 Solution A – Système japonais simplifié 19
6.2.2 Solution B – Système chinois 20
6.3 Gestion du suicide 20
6.3.1 Solution A – Autoriser le suicide 20
6.3.2 Solution B – Interdire le suicide 21
6.4 Gestion des annulations de coups 21
6.4.1 Solution A – Sans annulation 21
6.4.2 Solution B – Annulation avec historique complet 22
6.4.3 Solution C – Annulation limitée 22
6.5 Gestion des tailles du plateau 23
6.5.1 Solution A – Taille fixe (9x9) 23
6.5.2 Solution B – Multi-tailles (9x9, 13x13, 19x19) 23
6.6 Gestion du handicap 23
6.6.1 Solution A – Sans handicap 23
6.6.2 Solution B – Handicap automatique 24
6.7 Gestion du Komi 24
6.7.1 Solution A – Komi fixe 24
6.7.2 Solution B – Komi configurable 24
7. SOLUTION RECOMMANDÉE ET ARGUMENTATION 25
7.1 Choix des règles et fonctionnalités 25
7.2 Adéquation aux besoins du client 26
7.3 Simplicité de développement et de maintenance 26
7.4 Performance et contraintes techniques 26

7.5 Évolutivité de la solution 27
7.6 Argumentation globale 27
8. PLAN DE TRAVAIL ET ÉCHÉANCIER 28
8.1 Méthodologie de développement 28
8.2 Phases du projet 28
Livrables 28
Échéance 28
Livrable 1 : Cahier des charges – 10 avril 29
Phase 2 : Conception de la solution – 17 avril 29
Phase 3 : Développement et avancement – 17 avr.-26 mai 30
Phase 4 : Développement avancé et rencontres 30
Phase 5 : Produit final – 26 mai 30
Phase 6 : Guide utilisateur et bilan de projet – 26 mai 31
APPROBATIONS 31

1. CARACTÉRISATION DU CLIENT

1.1 Identification du client

Type de client :	Un groupe d'utilisateurs souhaitant jouer à un jeu de GO de manière simple et locale sur un ordinateur.
Taille du groupe cible :	2 joueurs par partie Le jeu étant conçu pour un mode local tour par tour, il est limité à deux joueurs utilisant le même appareil.

1.2 Profil des utilisateurs

Définition des utilisateurs :	Particuliers souhaitant jouer au jeu de GO de manière simple, rapide et intuitive, sans nécessiter de connaissances techniques avancées. Ils possèdent des compétences informatiques limitées et recherchent une application facile à utiliser.
Exemple de situation utilisateur :	Deux joueurs lancent une partie de GO. Ils entrent leur nom, sélectionnent les paramètres de la partie (taille du plateau, handicap, komi), puis jouent à tour de rôle. Le système gère automatiquement les règles, les tours, et affiche le résultat final (gagnant, score, temps, autre statistique pertinente).

1.3 Contexte organisationnel

L'application sera utilisée dans un contexte informel, généralement pour le divertissement.

Elle doit permettre de lancer rapidement une partie, jouer localement, afficher les résultats et sauvegarder certaines informations (noms, scores, temps, abandon).

L’application ne nécessite aucune connexion internet et fonctionne de manière autonome.

2. BESOIN ET CONTEXTE

2.1 Origine de la demande

Ce projet répond au besoin de créer un jeu de GO simple, accessible et fonctionnel, permettant à deux joueurs de jouer localement sans configuration complexe.

2.2 Problématique entraînant le besoin

Les difficultés rencontrées incluent :

- Absence d'un outil simple pour jouer au GO localement
- Complexité des logiciels existants
- Difficulté à gérer les règles pour les débutants
- Manque de clarté dans le suivi des parties (temps, score)
- Gestion manuelle des règles pouvant entraîner des erreurs

2.3 Cas d'utilisation type

Exemple concret :

- Deux joueurs lancent une partie de GO.
- Ils entrent leurs noms, sélectionnent un plateau 9x9, et commencent à jouer.
- Le système valide les coups, empêche les coups illégaux (ex : répétitions), et affiche le dernier coup joué.
- La partie se termine après deux passes consécutives ou abandon.

Le système affiche alors :

- Le gagnant
- Le score
- Le temps de jeu de chaque joueur

2.4 Liste de besoin explicites

L'application doit :

- Permettre d'entrer les noms des deux joueurs
- Permettre de créer une partie avec paramètres (taille, handicap, komi)
- Gérer un plateau de GO interactif
- Valider les coups (positions valides, éviter boucles infinies)
- Gérer les tours de jeu
- Afficher le dernier coup joué
- Calculer le score selon un système simple (inspiré du japonais)
- Permettre le passage de tour
- Détecter la fin de partie (2 passes ou abandon)
- Afficher un résultat clair (gagnant, score, temps)
- Gérer un chronomètre par joueur
- Sauvegarder certaines données (noms, score, temps, abandon)
- Interface simple et intuitive

2.5 Bénéfices attendus

- Facilité d'accès au jeu de GO
- Réduction des erreurs liées aux règles
- Expérience de jeu fluide
- Suivi simple des parties
- Application légère et rapide

3. ENVIRONNEMENT INFORMATIQUE

3.1 Plateforme cible

Système d'exploitation : Microsoft Windows (Version 10 et ultérieure)

Type d'application : Application native Windows (De bureau)

Architecture : Application sans serveur, avec enregistrement fichier local

3.2 Spécifications minimales du système

- Un ordinateur sous Windows ayant les spécifications minimales pour rouler le système d'exploitation (De ce même fait l'application en elle-même)
- Un espace de stockage suffisant pour un fichier de sauvegarde
- Un espace de stockage suffisant pour installer l'application

3.3 Environnement de développement

Langage de programmation : C# (.NET 8)

IDE: Microsoft Visual Studio Community

Framework UI : WPF (Windows Presentation Foundation)

Stockage des données : Fichiers XML (sans base de données)

4. ACTEURS DU SYSTÈME

4.1 Utilisateur

Rôle : Joueur 1 / Joueur 2

Description :

Personne qui joue une partie de GO.

- Entrer son nom
- Jouer des coups
- Passer son tour
- Consulter le résultat à la fin de la partie
- Consulter les détails de sa propre parite lorsqu'elle est jouée
- Crée une partie
- Modifier les paramètres initiaux de la partie qu'il joue
- Abandonner

5. FONCTIONNALITÉS DU SYSTÈME

5.1 Gestion des participants

5.1.1 Entrer les noms

Description : Entrer les noms de base de chacun des deux joueurs pour démarquer le joueur 1 et le joueur 2 textuellement.

Validations :

- Le nom ne peut pas être vide
- Le nom doit être unique dans le système
- Le nom ne doit pas contenir de caractères spéciaux invalides
- Le nom n'est pas trop long

5.2 Gestion des parties

5.2.1 Créer une partie

Description : Créer une nouvelle partie du jeu GO.

Données requises :

- Noms des joueurs
- Taille du plateau (9x9 par défaut)
- Handicap
- Komi (6.5 par défaut)
- Liste des participants impliqués

5.2.1 Terminer une partie

Description : Termine la partie en cours et calcule les données sommaires

Action requise :

- Deux passes consécutives/Abandon/Aucun coup possible

5.3 Gestion du plateau

5.3.1 Jouer un coup

Description : Placer une pierre sur la planche selon le tour du joueur.

Données requises :

- Position demandée
- Vérification de la validité du placement (Selon le tour du joueur, le suicide, le Ko, Captures, etc.)

5.3.2 Empêcher un coup illégal / Avertir

Description : Détecter les coups invalides, illogiques ou circulaires et les empêcher afin d'éviter un jeu

Données requises :

- Confirmation d'action
- Doit être effectuée depuis le compte ayant créée la dépense

5.4 Gestion du score et des règles

5.4.1 Calculer le score

Description :

Calculer automatiquement le score final de la partie selon un système inspiré des règles japonaises simplifiées.

Algorithme :

- Identifier toutes les zones vides du plateau
- Déterminer si une zone est entourée exclusivement par un seul joueur
- Attribuer les territoires au joueur correspondant
- Ajouter le komi au joueur blanc
- Calculer le score final de chaque joueur
- Déterminer le gagnant (score le plus élevé)

Précision :

Les calculs doivent être précis au demi-point près (ex : komi = 6.5).

5.4.2 Gestion des captures

Description :

Gérer automatiquement la capture des pierres adverses lorsque celles-ci n’ont plus de libertés.

Fonctionnement :

- Après chaque coup, vérifier les groupes de pierres adjacentes
- Déterminer si un groupe n’a plus de libertés
- Retirer les pierres capturées du plateau

Contraintes :

- La détection doit être immédiate après chaque coup
- Le système doit supporter les groupes de taille variable

5.4.2 Gestion des captures

Description :

Gérer automatiquement la capture des pierres adverses lorsque celles-ci n’ont plus de libertés.

Fonctionnement :

- Après chaque coup, vérifier les groupes de pierres adjacentes
- Déterminer si un groupe n’a plus de libertés
- Retirer les pierres capturées du plateau

Contraintes :

- La détection doit être immédiate après chaque coup
- Le système doit supporter les groupes de taille variable

5.4.3 Gestion des règles de jeu

Description :

Appliquer les règles essentielles du GO afin de garantir une partie valide.

Règles appliquées :

- Alternance des tours entre joueurs
- Interdiction de jouer sur une case occupée
- Détection des coups répétitifs (règle du Ko simple)
- Prévention des boucles infinies

Précision :

Le système doit privilégier une implémentation simple et robuste compatible avec les autres contraintes du projet.

5.4.4 Gestion du suicide

Description :

Permettre (temporairement) les coups suicides pour simplifier l'implémentation initiale.

Comportement :

- Le système autorise le coup
- Une détection peut être effectuée pour avertir l'utilisateur
- Possibilité d'interdire ces coups dans une version future

5.4.5 Fin de partie

Description :

Déterminer automatiquement la fin d'une partie.

Conditions :

- Deux passes consécutives
- Abandon d'un joueur
- Indisponibilité de jouer

Comportement :

- Si abandon : le joueur abandonneur perd automatiquement
- Sinon : calcul du score final

5.4.6 Affichage du résultat

Description :

Afficher un rapport clair de fin de partie.

Contenu du rapport :

- Nom des joueurs
- Score final de chaque joueur
- Gagnant
- Temps de jeu individuel
- Temps total de la partie
- Indication d'abandon (si applicable)

5.5 Gestion du temps

5.5.1 Chronomètre des joueurs

Description :

Mesurer le temps de jeu de chaque joueur individuellement.

Fonctionnement :

- Un chronomètre est associé à chaque joueur
- Le temps s'incrémente uniquement lorsque c'est son tour
- Le chronomètre s'arrête automatiquement lors du changement de tour

5.5.2 Temps total de la partie

Description :

Mesurer la durée totale de la partie.

Fonctionnement :

- Démarre au premier coup
- S'arrête à la fin de la partie

5.5.3 Sauvegarde des temps (en même temps que d'autres données)

Description :

Enregistrer les temps dans le fichier de données.

Données sauvegardées :

- Temps joueur 1
- Temps joueur 2
- Temps total

5.6 Gestion des données

5.6.1 Sauvegarder les données

Description :

Enregistrer les données essentielles d'une partie dans un fichier XML.

Format :

XML valide et structuré

Données sauvegardées :

- Nom du joueur 1
- Nom du joueur 2
- Score final
- Temps individuels
- Temps total
- Indication d'abandon

Précision :

- Aucune sauvegarde des coups ou du plateau
- Le fichier est réécrit à chaque nouvelle partie

5.6.2 Récupération des données

Description :

Non applicable dans cette version. Au stade actuel de l'application, seul l'écriture de données persistentes est implémentée, par la demande du client.

Précision :

- Les données ne sont pas rechargées
- L'application démarre toujours avec une nouvelle partie

6. FAISABILITÉ DES SOLUTIONS PROPOSÉES

Plusieurs choix de conception ont été analysés afin de déterminer les règles et fonctionnalités à implémenter dans le jeu de GO. Chaque option présente des avantages et des contraintes en fonction de la complexité de développement et des objectifs du projet.

6.1 Gestion de la règle du Ko

6.1.1 Solution A – Ko simple

Description :
Empêche uniquement la répétition immédiate d'un état précédent du plateau.

Avantages :

- Facile à implémenter
- Peu coûteux en mémoire
- Suffisant pour éviter les boucles simples

Inconvénients :

- Ne couvre pas toutes les situations complexes
- Peut permettre certaines répétitions indirectes

6.1.2 Solution B – Superko

Description :

Empêche toute répétition d'un état précédent du plateau, peu importe le moment où il est survenu.

Avantages :

- Conforme aux règles avancées du GO
- Empêche toutes les répétitions possibles

Inconvénients :

- Complexité élevée
- Nécessite de stocker tous les états du plateau
- Augmente l'utilisation mémoire

6.2 Gestion des règles de score

6.2.1 Solution A – Système japonais simplifié

Description :

Calcul basé sur les territoires contrôlés, avec ajout du komi.

Avantages :

- Plus simple à implémenter
- Suffisant pour une expérience de jeu fonctionnelle
- Cohérent avec les attentes du projet
- Inconvénients :
- Moins précis que certaines variantes complètes
- Ne prend pas en compte tous les cas avancés

Inconvénients :

- Moins précis que certaines variantes complètes
- Ne prend pas en compte tous les cas avancés

6.2.2 Solution B – Système chinois

Description :

Calcul basé sur les territoires et les pierres présentes sur le plateau.

Avantages :

- Plus simple à calculer dans certains cas
- Moins ambigu pour certaines positions

Inconvénients :

- Différent des règles classiques attendues
- Peut être déroutant pour certains utilisateurs

6.3 Gestion du suicide

6.3.1 Solution A – Autoriser le suicide

Description :

Permettre aux joueurs de placer une pierre qui n’a pas de libertés.

Avantages :

- Simplifie l’implémentation
- Réduit la complexité des validations

Inconvénients :

- Non conforme aux règles standards
- Peut créer des situations incohérentes

6.3.2 Solution B – Interdire le suicide

Description :

Empêcher le placement de pierres sans libertés.

Avantages :

- Conforme aux règles officielles
- Évite les erreurs de jeu

Inconvénients :

- Complexité de validation supplémentaire
- Nécessite une détection complète des libertés

6.4 Gestion des annulations de coups

6.4.1 Solution A – Sans annulation

Description :

Aucune possibilité de revenir en arrière.

Avantages :

- Implémentation simple
- Aucun stockage d’état nécessaire

Inconvénients :

- Expérience utilisateur limitée
- Aucune tolérance aux erreurs

6.4.2 Solution B – Annulation avec historique complet

Description :

Permettre de revenir à un état précédent en sauvegardant chaque coup.

Avantages :

- Très flexible
- Améliore l'expérience utilisateur

Inconvénients :

- Nécessite de stocker chaque état du plateau
- Complexité élevée
- Augmentation mémoire

6.4.3 Solution C – Annulation limitée

Description :

Permettre d'annuler uniquement le dernier coup.

Avantages :

- Compromis entre simplicité et utilité
- Moins coûteux que l'historique complet

Inconvénients :

- Moins flexible
- Toujours nécessite un stockage minimal

6.5 Gestion des tailles du plateau

6.5.1 Solution A – Taille fixe (9x9)

Avantages :

- Implémentation simple
- Adaptée au projet

Inconvénients :

- Limitation des options

6.5.2 Solution B – Multi-tailles (9x9, 13x13, 19x19)

Avantages :

Plus flexible

Plus proche du GO réel

Inconvénients :

- Complexité accrue
- Plus de tests requis

6.6 Gestion du handicap

6.6.1 Solution A – Sans handicap

Avantages :

- Simple

Inconvénients :

- Déséquilibre possible entre joueurs

6.6.2 Solution B – Handicap automatique

Description :

Placement automatique des pierres selon des positions standard.

Avantages :

- Conforme aux règles du GO
- Équilibre les parties

Inconvénients :

- Implémentation supplémentaire
- Nécessite gestion des positions prédéfinies

6.7 Gestion du Komi

6.7.1 Solution A – Komi fixe

Avantages :

- Simple

Inconvénients :

- Moins flexible

6.7.2 Solution B – Komi configurable

Avantages :

- Flexible
- Plus réaliste

Inconvénients :

- Complexité supplémentaire

7. SOLUTION RECOMMANDÉE ET ARGUMENTATION

À la suite de l'analyse des différentes options de conception, une solution globale a été retenue afin d'assurer un équilibre entre simplicité de développement, robustesse et qualité de l'expérience utilisateur.

Cette solution vise à implémenter un jeu de GO fonctionnel, stable et évolutif, tout en respectant les contraintes du projet.

7.1 Choix des règles et fonctionnalités

Les choix suivants ont été retenus pour l'implémentation du système :

- **Règle du Ko :** utilisation du Ko simple
- **Système de score :** système japonais simplifié basé sur les territoires
- **Suicide :** autorisé dans la version initiale
- **Annulation de coup (undo) :** non implémentée
- **Tailles de plateau :**
  - 9x9 entièrement fonctionnel
  - 13x13 et 19x19 visibles mais non activés
- **Handicap :** implémenté avec placement automatique selon positions standards
- **Komi :** valeur par défaut de 6.5, variable prévue mais partiellement implémentée
- **Mode de jeu :** local, deux joueurs sur le même appareil
- **Sauvegarde :** uniquement les résultats (noms, score, temps, abandon), sans historique des coups

7.2 Adéquation aux besoins du client

La solution retenue répond directement aux besoins identifiés, soit :

- Offrir une expérience de jeu simple et accessible
- Permettre une prise en main rapide sans apprentissage complexe
- Garantir le respect des règles essentielles du GO
- Fournir un système fiable de gestion des parties (tour, score, fin de partie)

Le choix d'une implémentation progressive permet également de prioriser les fonctionnalités essentielles tout en laissant place à des améliorations futures.

7.3 Simplicité de développement et de maintenance

La solution privilégie une approche modulaire et progressive :

- Les règles complexes sont simplifiées lorsque possible
- Les fonctionnalités non essentielles sont reportées ou limitées
- L'architecture est conçue pour permettre des extensions futures

Cette approche permet :

- De réduire les risques d'erreurs
- De faciliter le débogage
- D'assurer une meilleure stabilité globale

7.4 Performance et contraintes techniques

Les choix effectués permettent de limiter :

- L'utilisation mémoire
- La complexité des algorithmes
- Les besoins en stockage

Cela garantit une application légère, rapide et adaptée à une exécution locale sans dépendances externes.

7.5 Évolutivité de la solution

L'application est conçue de manière à pouvoir évoluer facilement :

- Support futur de nouvelles tailles de plateau
- Amélioration des règles de jeu
- Ajout de fonctionnalités avancées (undo, règles complètes, etc.)

Cette flexibilité permet d'améliorer progressivement le produit sans devoir modifier l'architecture de base.

7.6 Argumentation globale

La solution retenue est particulièrement adaptée au contexte du projet :

- Répond aux besoins essentiels sans surcomplexifier le développement
- Permet de livrer une application fonctionnelle dans les délais
- Offre une base solide pour des améliorations futures
- Garantit une expérience utilisateur claire et cohérente

En priorisant la simplicité, la robustesse et l'évolutivité, cette approche assure un équilibre optimal entre qualité du produit et faisabilité technique.

8. PLAN DE TRAVAIL ET ÉCHÉANCIER

8.1 Méthodologie de développement

Le projet sera réalisé selon une approche **itérative et incrémentale** inspirée des méthodologies agiles.

Chaque itération permettra de développer une portion fonctionnelle du jeu, testée et validée avant de passer à la suivante.

Cette approche permet :

- D'identifier rapidement les problèmes techniques
- De valider progressivement les fonctionnalités critiques
- D'adapter le développement en fonction des contraintes rencontrées

Les éléments les plus complexes (gestion du plateau, captures, validation des coups) seront développés en priorité afin de sécuriser la base du projet.

8.2 Phases du projet

Livrables	Échéance
Cahier des charges fonctionnel	10 avril
Conception détaillée de la solution	17 avril
Application - État d'avancement	 17-26 mai.

- Produit final - Plusieurs rencontres clients lors du projet	26 mai Dès le 13 avril
Guide d'utilisateur	26 mai
Bilan de projet (post-mortem)	26 mai

Livrable 1 : Cahier des charges – 10 avril

Activités :

- Compréhension du projet et des objectifs
- Analyse des règles du jeu de GO
- Identification des fonctionnalités nécessaires
- Élaboration des choix de conception (règles, limitations, fonctionnalités)
- Rédaction du cahier des charges

Livrable :

- Document complet décrivant les besoins, fonctionnalités et contraintes

Phase 2 : Conception de la solution – 17 avril

Activités :

- Conception de l'interface utilisateur (plateau, menus, affichage)
- Modélisation orientée objet :
- Classe Game (gestion globale)
- Classe Board (plateau)
- Classe Player (joueur)
- Définition des structures de données du plateau
- Conception des algorithmes principaux :
- Détection des libertés
- Gestion des captures
- Validation des coups
- Calcul du score
- Élaboration des tests unitaires et fonctionnels

Livrable (🡐 Phase 2):

- Schémas de conception
- Description des algorithmes

Phase 3 : Développement et avancement – 17 avr.-26 mai

Activités :

- Mise en place de l'environnement WPF
- Implémentation du plateau de jeu (grille dynamique)
- Gestion de l'entrée des joueurs (noms)
- Implémentation de la pose de pierres
- Alternance des tours entre joueurs

Objectif :
Obtenir une version jouable de base sans règles avancées

Phase 4 : Développement avancé et rencontres

Activités :

- Implémentation de la gestion des captures
- Détection des libertés des groupes de pierres
- Validation des coups (cases occupées, répétitions)
- Implémentation de la règle du Ko simple
- Gestion des passes et de la fin de partie
- Ajout du système de handicap
- Implémentation du chronomètre des joueurs

Objectif :
Obtenir un jeu fonctionnel respectant les règles principales

Phase 5 : Produit final – 26 mai

Activités :

- Implémentation du calcul du score final
- Ajout du komi
- Affichage du résultat (gagnant, score, temps)
- Amélioration de l'interface utilisateur
- Tests complets (fonctionnels et intégration)
- Correction des bugs

Objectif :

Application complète, stable et prête à être livrée

Phase 6 : Guide utilisateur et bilan de projet – 26 mai

Activités :

- Rédaction du guide d'installation
- Rédaction du guide d'utilisation
- Vérification de la clarté des instructions
- Validation interne du produit final
- Analyse des forces et faiblesses du projet
- Documentation des apprentissages

Livrables :

- Guide utilisateur
- Bilan de projet (post-mortem)

APPROBATIONS

Client	Chef de projet
Signature : _____ Date : _____	Signature : _____ Signature : _____ Date : _____

ANNEXE 1
Questions client

Voici la liste de questions permettant d'assurer la qualité et la bonne fonction du jeu mandaté GO :

Le jeu doit-il inclure un système de **connexion utilisateur (login)** et de gestion de comptes?

Nom pseudo pour deux joueurs, joueur gagnant (écriture seulement), pointages des deux personnes et temps.

Souhaite-t-on intégrer un **système de classement (ranking / leaderboard)** pour les joueurs?

Non, mais pointage par partie

L'application doit-elle démarrer avec un **menu principal**, ou **ouvrir directement le menu de nouvelle partie**?

Si nécessaire

Existe-t-il une **image de marque (logo, couleurs, identité visuelle)** à appliquer dans l'application?

Cégép Chicoutimi

L'application doit-elle supporter **plusieurs langues** et permettre le **changement de langue**?

Non

Doit-on offrir **plusieurs thèmes visuels** (clair, sombre, etc.)?

Pas besoin, voir Material Design si tu veux

L'application doit-elle inclure des options d'**accessibilité** (daltonisme, malvoyance, etc.)?

Non

Le jeu doit-il **redimensionner dynamiquement le plateau**, ou simplement supporter **plein écran / petit écran**?

La fenêtre peut être redimensionnée, mais pas obligée de faire suivre la taille de la board.

Doit-on permettre de jouer :

- contre un **robot**
- et/ou en **local** (2 joueurs sur le même appareil)
- et/ou en **multijoueur en ligne**

Local tour par tour

Pour les parties jouées en multijoueur et/ou en local, doit-on **sauvegarder des paramètres sur les utilisateurs, et le résultat des parties** ?

Voir 1. On ne lit pas le fichier alors lorsqu'on démarre une partie, on ré-écrit les noms. On ne sauvegarde pas les coups ni les planches au complet.

Pour les parties **locales ou contre un robot**, doit-on **sauvegarder chaque coup** afin de permettre de **quitter et reprendre une partie**?

Voir 1

Doit-on permettre un **mode de jeu alterné**, où une partie peut rester **en attente du coup de l'adversaire,** et le joueur peut lancer plusieurs parties ?

Non

Doit-on permettre différentes **variantes de règles** (ex. règles de **New Zealand**, où le suicide est légal) ?

Non

Pour lisntant, permettre suicide par simplicité, mais avertir user si ca arrive?? (détecter possible)

À la fin pour polir, implémente le si tu as le temps

Quel **système de pointage** doit être utilisé?

Options possibles :

- **Système japonais / coréen** (captures, territoires, komi)
- **Système chinois** (territoires + pierres sur le plateau)
- **American Go Association**

Japonais, pas besoin de compter pierres (prendre le plus simple)

Doit-on permettre plusieurs **tailles de plateau**? Si tel est le cas, est-elle prédéfinie ou complètement personnalisable ?

Exemple d'options possibles :

- 19 × 19 (standard)
- 13 × 13
- 9 × 9

Mettre les trois, mais implémenter parfaitement 9x9 seulement. 13x13 et 19x19 visibles mais not enabled YET. **MAIS le développement doit être dynamique pour implementer PLUS TARD**

Le jeu doit-il permettre les **pierres de handicap** pour équilibrer les parties entre joueurs de niveaux différents?

Sélectionner handicap lors de création de partie, jeu place aléatoirement les handicaps selon des possibilités de positions pré-établies standard

Le **komi** (points ajoutés au joueur blanc pour compenser l'avantage du noir) doit-il être :

- fixe
- configurable
- inexistant

Prévoir variable du komi, default 6.5 et implémenter partiellement. A la fin, implémenter au complet si tu as le temps.

Le système doit-il **empêcher automatiquement les coups illégaux** (ko, suicide interdit, etc.)?

Interdir boucle infinie etc. (coups successifs a même place et autres coups illogiques)

Quelle **variante de la règle du Ko** doit être appliquée?
Options possibles :

- Ko simple
- Superko

Choisir le plus simple et qui ne contredit pas avec les spcéficiations.

Le jeu doit-il **afficher le dernier coup joué** (ex. cercle ou surbrillance)?

Oui, idéalement.

Doit-on permettre de **revoir l'historique des coups** et naviguer dans la partie?

Non

Le joueur peut-il **annuler un coup (undo)**?

- seulement contre l'IA
- aussi en multijoueur

Voir si possibilité de undo, car pas de sauvegarde de board a chaque state. Implémenter si c'est possible et faisable.

Doit-on inclure un **système de temps par joueur**?

Chrono pour référence seulement. Avec temps individuels des participants et temps total aussi. Chacun doit avoir un chronomètre qui s'augmente lorsque c'est à leur tour. **Sauvegarder TEMPS joueur 1 et joueur 2**

Comment la **fin de partie** est-elle déterminée?

Deux Pass consécutives ou

abandon d'un joueur (si implémenté) : Si abandon, abandonneur perd

Doit-on permettre **l'export ou l'import de parties** dans un format standard (ex. SGF, csv)?

Non

Le jeu doit-il inclure un **tutoriel ou mode apprentissage** pour les nouveaux joueurs?

Non

Doit-on enregistrer les **statistiques du joueur**?
(Victoires, Défaites, Ratio, Abandons…)

Enregistrer abandon dans fichier (qui invalide le nb de points du joueur dans la partie) Aucune valeur calculée dans le fichier. Sinon, voir 1